UNIVALI – Universidade do Vale do Itajaí Disciplina: Arquitetura e Organização de Computadores
Prática 01A – Programação em linguagem de montagem

<u>Lei atentamente as instruções abaixo</u>:

1. Esta atividade ser feita em duplas ou trios.

2. A atividade consiste na implementação de **um programa** utilizando a linguagem de montagem do RISC-V32i (conforme as instruções a seguir).

3. *Os arquivos ASM de cada questão devem ser postados no AVA da disciplina em um único arquivo .ZIP até o final da aula.*

4. Cada código-fonte deve ser escrito em um arquivo com extensão .asm e, **obrigatoriamente**, com um nome que identifique o desenvolvedor e o nome do programa (ex. ***RingoStarr_Programa_01.asm***).
5. Cada código fonte deve conter um cabeçalho comentado que identifique a disciplina, a atividade, o programa e o nome desenvolvedor. Ex:

Disciplina: Arquitetura e Organização de Processadores
Atividade: Avaliação 01 – Programação em Linguagem de Montagem
Programa 01
Nome: Nome do Autor

6. Não serão aceitos trabalhos entregues em atraso.

7. O professor poderá solicitar que **o aluno** faça uma **demonstração explicativa da execução dos códigos no RARS**.

8. **A implementação deverá apresentar resultados corretos para qualquer conjunto de dados.** Uma solução que **não execute corretamente** terá, automaticamente, um **desconto de 50% na nota**, sendo que o professor também avaliará a correção de segmentos específicos do código (controle de execução, acesso a memória,...).

9. Se forem identificados **trabalhos** com grau de **similaridade** que caracterize cópia (autorizada ou não) ou adaptação, a nota dos grupos será a **nota de um trabalho dividida** pelo número de grupos que entregou esses trabalhos similares.

10. Cada uso de pseudo-instrução, com exceção da instrução `la`, implicará num desconto de 0.5 na nota.

CÓDIGOS DE LEITURA E ESCRITA BÁSICOS

```
.data
    string1: .asciz "Informe um valor: "
.text
    #comandos para escrever a string1 no console
    addi a7, zero, 4 #informa a operação de escrita te texto
    la   a0, string1 #coloca o endereço da string em a0
    ecall            #executa a operação

    #comandos para ler um inteiro do console
    addi a7, zero, 5 #informa a operação de leitura de inteiro
    ecall            #executa a operação
    add s0, zero, a0 #transfere o valor lido de a0 para s0

    #comandos para escrever um inteiro do console
    addi a7, zero, 1 #informa a operação de escrita de inteiro
    add a0, zero, s0 #coloca em a0 o inteiro que será escrito
    ecall            #executa a operação
```

CÓDIGOS DE DESVIO BÁSICOS

```
beq s0, s1, igual #se (s0 == s1) devia para igual
igual:
bne s0, s1, diferente #se (s0 != s1) devia para diferente
diferente:
bge  s0, s1, maior_igual #se (s0 >= s1) devia para maior_igual
maior_igual:
blt  s0, s1, menor #se (s0 < s1) devia para menor
menor:

#EXEMPLO: if(s0 == s1 and s0 == s2) {}
bne s0, s1, diferente
bne s0, s2, diferente
    #só entra aqui se forem iguais, senão vai ter desviado por serem diferentes
diferente:

#EXEMPLO: if(s0 == s1) {} else {}
bne s0, s1, diferente
    #se forem iguais não desvia
    jal zero, fim_if1 # esse comando faz saltar o else, pois não queremos executá-lo
diferente:
    #essa é a condição do else
fim_if1:
```

CÓDIGOS DE LAÇO DE REPETIÇÃO BÁSICO

```
#int t1=10;
#while(t1<t0){
#   t0++;
#}
addi t1,zero, 10
while:
  bge t0, t1, fim_while1    #t0<t1
  addi t0,t0,1              #t0++
  jal zero, while           #recomeça o laço
fim_while1:
nop                         #no operation
```

CÓDIGOS LEITURA E ESCRITA EM MEMÓRIA BÁSICOS

```
.data
    memoria1: .word 10 #vetor com apenas uma posição
.text

    #comando para ler da memória e colocar em s0
    la t0, memoria1 #coloca o início da memoria1 em t0
    lw s0, 0(t0)    #s0 = t0[0]

    #comando para colocar o valor 20 na memoria1
    addi s1, zero, 20
    la t0, memoria1 #coloca o início da memoria1 em t0
    sw s1, 0(t0)    #t0[0] = s1
```

PROGRAMA 01 (5,0 pontos)

Enunciado:

Considere uma disciplina com 16 dias de aulas e que é necessário registrar a presença de cada aluno em cada dia, consumindo o mínimo de variáveis em memória. Assumindo que a turma tenha no máximo 32 alunos, implemente um programa que utilize um vetor de 16 elementos inteiros (um elemento para cada dia) para registrar a frequência dos alunos de cada disciplina e que ofereça ao professor funções para registrar a presença e ausência de qualquer aluno em qualquer dia.

Leia atentamente cada requisito listado abaixo, pois o atendimento deles será considerado para fins de avaliação (o não atendimento de algum requisito implica em desconto de nota).

Requisitos:

1. **Utilizar pseudo instruções** com exceção do comando `la` que é uma pseudo instrução que não aprendemos a contornar ainda, **implicará num desconto de 0.5 por instrução**.

2. Na seção de declaração de variáveis (`.data`), declare um vetor com 16 elementos inicializados com todos os seus bits em 1 para assumir, por padrão, que todos os alunos estão presentes. A forma mais fácil de fazer isso é utilizar a notação hexadecimal para representar valores com todos os bits em 1 (0xFFFFFFFF).

3. O programa deve ficar em laço infinito solicitando ao usuário para indicar um dia de aula, o aluno a ser registrado e o registro a ser feito (presença ou ausência) e então realizar as operações indicadas nos itens a seguir. Na interface, utilizes mensagens como estas:

    a. Entre com o número da aula (de 0 a 15):
    b. Entre com o número do aluno (de 0 a 31):
    c. Entre com o tipo do registro (presença = 1; ausência = 0):

4. Após a leitura dos dados, dentro do laço, o programa deve construir uma máscara de bits adequada para realizar a operação solicitada (registro de presença ou de ausência) sobre o bit do aluno indicado no dia indicado. Exemplos:

    a. Para registrar a ausência do aluno de número 2, deve ser construída uma máscara de 32 bits apropriada para desligar o bit correspondente (bit 2) e manter os demais inalterados.
    b. Para registrar a presença do aluno de número 20, deve ser construída uma máscara de 32 bits apropriada para desligar o bit correspondente (bit 17) e manter os demais inalterados.

    NOTA: Para construir a máscara de bits, será necessário utilizar operações lógicas de deslocamento e de manipulação de bit.

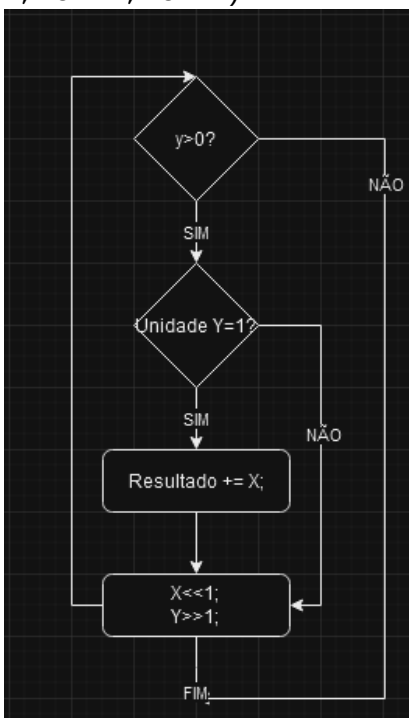
5. Ainda dentro do laço, o programa deve realizar a operação lógica correspondente ao registro solicitado pelo usuário e usando a máscara de bits construída previamente.

6. Os programas devem ser escritos respeitando o estilo de programação ASM, usando tabulação para organizar o código em colunas (rótulos, mnemônicos, operandos e comentários).

7. Procure comentar ao máximo o seu código. Isto é um hábito da programação *assembly*.

PROGRAMA 02 (5,0 pontos)

Enunciado:
Implemente um programa que solicite 2 valores inteiros ao usuário. O programa deverá implementar a multiplicação desses dois valores apenas executando operações de soma (`add`, `addi`) e deslocamento (`sll, srl, slli, srli`)



Requisitos:

1. O usuário deverá solicitar dois valores: X e Y;

2. O programa deverá realizar o mínimo de cálculos possíveis, parando de calcular quando o denominador chegar em zero.

3. O programa deverá constantemente atualizar os valores de X e Y, com seus deslocamentos a esquerda e a direita conforme mostrado no diagrama de sequência.

4. Ao término, o programa deverá exibir o resultado.